SEARCHING AND HASHING

Unit 7

Presented By: Saroj Bhandari

Syllabus

- 7.1 Introduction to Searching, Search Algorithms: Sequential Search, Binary Search
- 7.2 Efficiency of Search Algorithms
- 7.3 Hashing : Hash Function and Hash Tables, Collision Resolution Techniques

Searching Algorithm

- **Searching algorithms** are essential tools in computer science used to locate specific items within a collection of data. In this tutorial, we are mainly going to focus upon searching in an array. When we search an item in an array, there are two most common algorithms used based on the type of input array.
- **Linear Search** : It is used for an unsorted array. It mainly does one by one comparison of the item to be search with array elements. It takes linear or O(n) Time.
- **Binary Search** : It is used for a sorted array. It mainly compares the array's middle element first and if the middle element is same as input, then it returns. Otherwise it searches in either left half or right half based on comparison result (Whether the mid element is smaller or greater). This algorithm is faster than linear search and takes O(Log n) time.

Essential of search

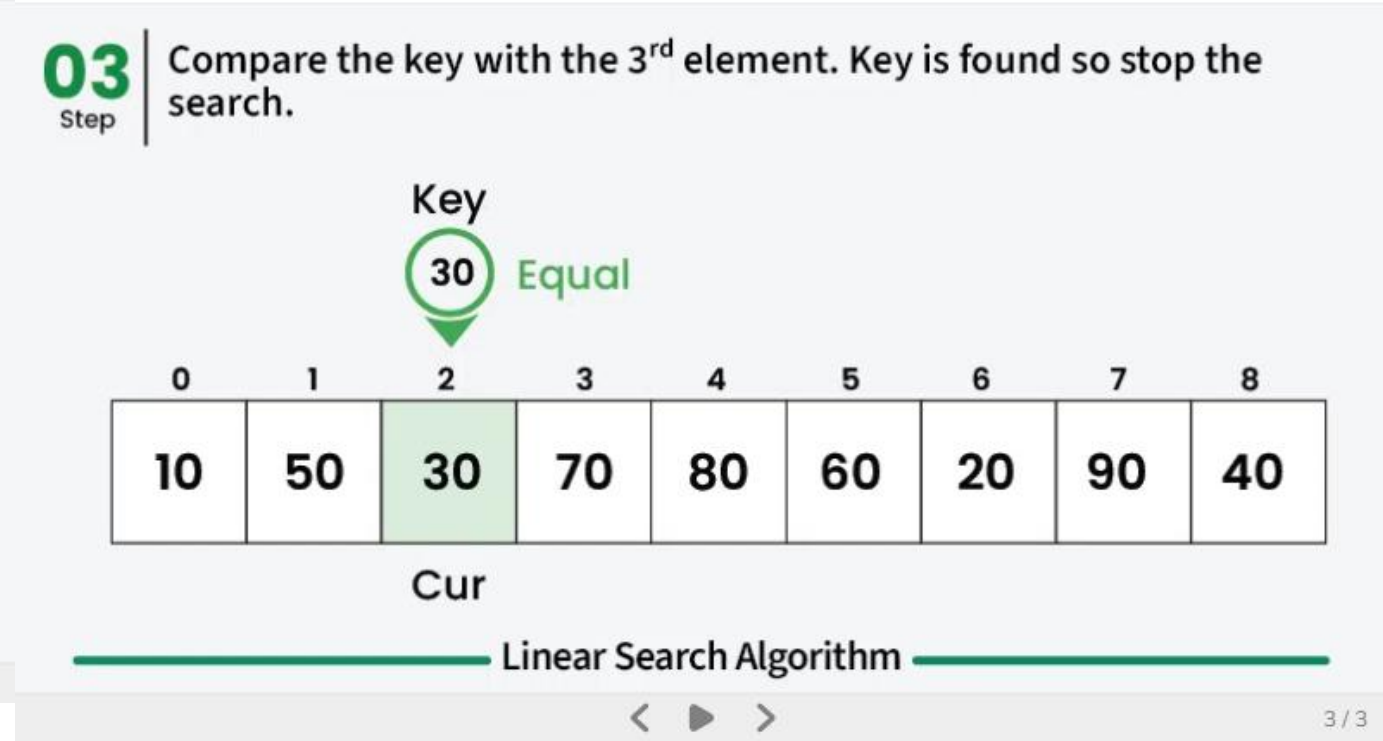
- Searching technologies are essential for efficiently locating specific data within large datasets, making them fundamental for computer science and data processing. They enable quick retrieval of information, improve decision-making, and enhance various aspects of daily life.
- Key Aspects of Searching Technologies:
- **Efficiency:**
  - Searching algorithms are designed to minimize the time required to find data, especially in large datasets.
- **Data Organization:**
  - Search technologies often rely on indexing and organizing data in a way that allows for fast retrieval, according to ScienceDirect.com.
- **Algorithm Variety:**
  - Different searching algorithms (e.g., linear search, binary search) are used depending on the data structure and the specific needs of the search.

Linear Search Algorithm

- Sequential search, also known as linear search, is a simple algorithm for finding a specific item within a list.
- It works by examining each element of the list one by one in order until a match is found or the entire list has been checked.
- Given an array, **arr** of n integers, and an integer element **x**, find whether element **x** is present in the array. Return the index of the first occurrence of **x** in the array, or -1 if it doesn't exist.
- ***Input**: arr[] = [1, 2, 3, 4], x = 3*
  ***Output**: 2*
  ***Explanation**: There is one test case with array as [1, 2, 3 4] and element to be searched as 3. Since 3 is present at index 2, the output is 2.*

For example

- Consider the array **arr[] = {10, 50, 30, 70, 80, 20, 90, 40}** and **key** = 30



Code

```c
// C code to linearly search x in arr[].

#include <stdio.h>

int search(int arr[], int N, int x)
{
    for (int i = 0; i < N; i++)
        if (arr[i] == x)
            return i;
    return -1;
}

// Driver code
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int N = sizeof(arr) / sizeof(arr[0]);

    // Function call
    int result = search(arr, N, x);
    (result == -1)
        ? printf("Element is not present in array")
        : printf("Element is present at index %d", result);
    return 0;
}
```

Time and Space Complexity of Linear Search Algorithm:

- **Time Complexity:**
  - **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is O(1)
  - **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is O(N) where N is the size of the list.
  - **Average Case:** O(N)
- **Auxiliary Space:** O(1) as except for the variable to iterate through the list, no other variable is used.

Applications of Linear Search Algorithm:

- **Unsorted Lists:** When we have an unsorted array or list, linear search is most commonly used to find any element in the collection.
- **Small Data Sets:** Linear Search is preferred over binary search when we have small data sets with
- **Searching Linked Lists:** In linked list implementations, linear search is commonly used to find elements within the list. Each node is checked sequentially until the desired element is found.
- **Simple Implementation:** Linear Search is much easier to understand and implement as compared to Binary Search or Ternary Search.

When to use Linear Search Algorithm?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

Advantages and Disadvantage

Advantage:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Disadvantages :

- Linear search has a time complexity of O(N), which in turn makes it slow for large datasets.
- Not suitable for large arrays.

Binary Search Algorithm

- **Binary Search Algorithm** is a searching algorithm used in a sorted array by **r**epeatedly dividing the search interval in half.
- The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(log N).

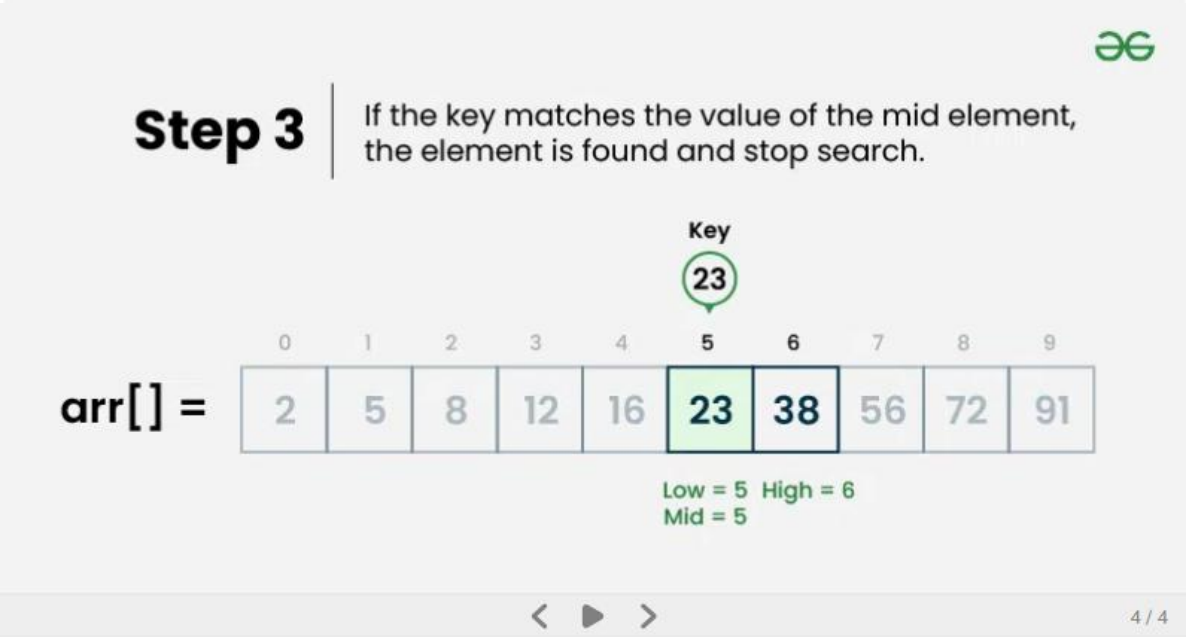
- **Conditions to apply Binary Search Algorithm in a Data Structure**
  - To apply Binary Search algorithm:
  - The data structure must be sorted.
  - Access to any element of the data structure should take constant time.

Binary Search Algorithm

- Divide the search space into two halves by **finding the middle index "mid"**.
- Compare the middle element of the search space with the **key**.
- If the **key** is found at middle element, the process is terminated.
- If the **key** is not found at middle element, choose which half will be used as the next search space.
  - If the **key** is smaller than the middle element, then the **left** side is used for next search.
  - If the **key** is larger than the middle element, then the **right** side is used for next search.
- This process is continued until the **key** is found or the total search space is exhausted.

How does Binary Search Algorithm work?

- To understand the working of binary search, consider the following illustration:
- Consider an array **arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}**, and the **target = 23**.



Code

```c
// C program to implement iterative Binary Search
#include <stdio.h>

// An iterative binary search function.
int binarySearch(int arr[], int low, int high, int x)
{
    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Check if x is present at mid
        if (arr[mid] == x)
            return mid;

        // If x greater, ignore left half
        if (arr[mid] < x)
            low = mid + 1;

        // If x is smaller, ignore right half
        else
            high = mid - 1;
    }

    // If we reach here, then element was not present
    return -1;
}
```

```c

// Driver code
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;
    int result = binarySearch(arr, 0, n - 1, x);
   if(result == -1) printf("Element is not present in array");
   else printf("Element is present at index %d",result);

}
```

Time Complexity: O(log N)
Auxiliary Space: O(1)

Complexity Analysis of Binary Search Algorithm

- **Time Complexity:**
  - Best Case: O(1)
  - Average Case: O(log N)
  - Worst Case: O(log N)
- **Auxiliary Space:** O(1), If the recursive call stack is considered then the auxiliary space will be O(log N).

Applications of Binary Search Algorithm

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- It can be used for searching in computer graphics such as algorithms for ray tracing or texture mapping.
- It can be used for searching a database.

Binary Vs Linear Search

Feature	Linear Search	Binary Search
Definition	Checks each element one by one	Divides the sorted list in half repeatedly
Data Requirement	Works on **unsorted or sorted** data	Works **only on sorted** data
Time Complexity		
- Best Case	O(1)	O(1)
- Average Case	O(n)	O(log n)
- Worst Case	O(n)	O(log n)
Space Complexity	O(1)	O(1) (iterative) / O(log n) (recursive)
Ease of Implementation	Very easy	Slightly more complex
Performance	Slower on large datasets	Faster on large sorted datasets
Use Case	Small or unsorted datasets	Large and sorted datasets

Presented By: Saroj Bhandari

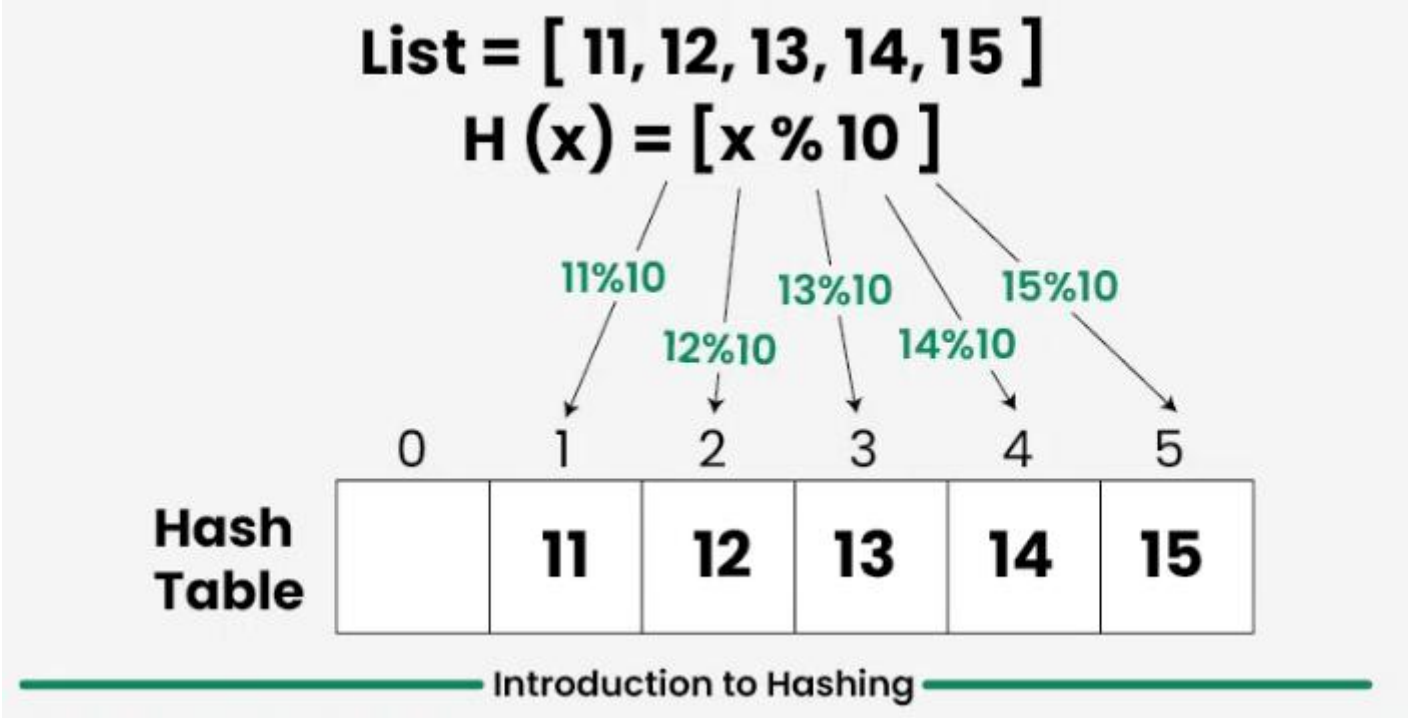
Searching Algorithms:

- **Blind search algorithms,** also known as uninformed search algorithms, are a class of algorithms that explore a search space without any knowledge about the likely direction to the goal state.
- They systematically explore the search tree, using techniques like Breadth-First Search (BFS) or Depth-First Search (DFS).
- **Best First Search (BFS)** is a pathfinding algorithm that uses a heuristic to evaluate and prioritize nodes in a graph, expanding the most promising one first.
- It's an informed search algorithm that aims to find the optimal path by leveraging a heuristic function that estimates the cost to reach the goal from a given node.
- Example: A Search*
  - A* search is a well-known example of a Best First Search algorithm. It uses a heuristic function that estimates the distance to the goal and also considers the cost of moving from the current node to its neighbors. A* is often used in pathfinding problems because it balances exploring promising paths with ensuring optimality, according to a post on YouTube.

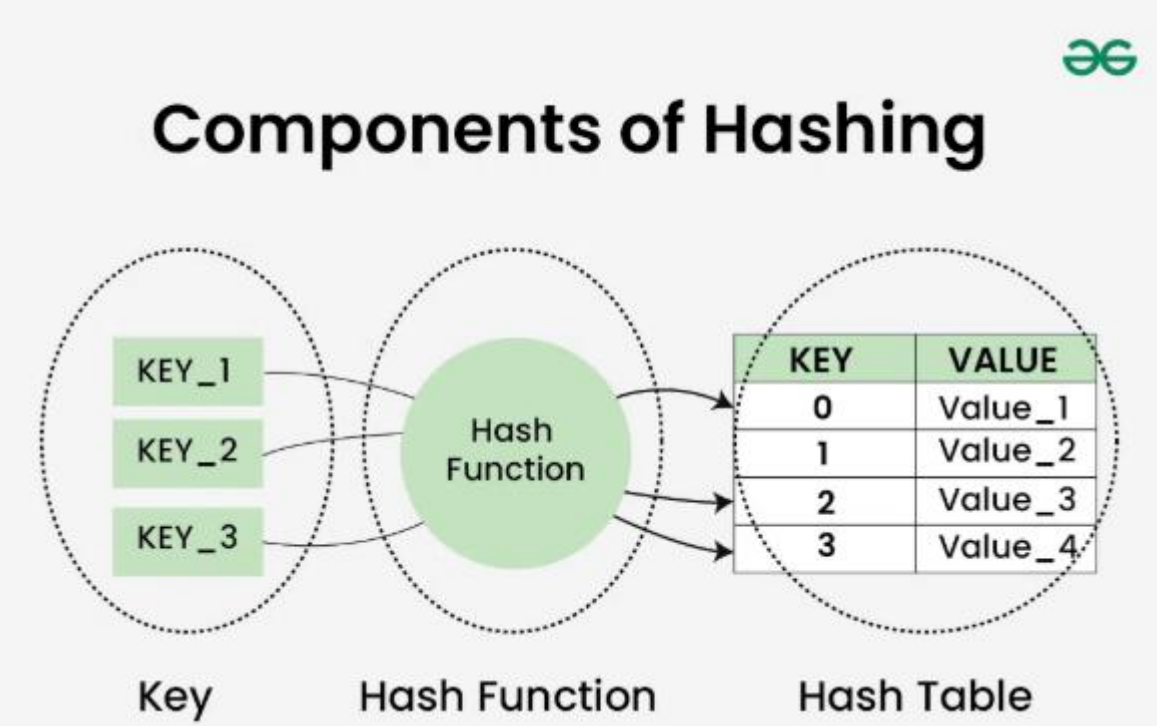
Hashing in Data Structure

- **Hashing** is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access.
- Hashing involves mapping data to a specific index in a hash table (an array of items) using a **hash function.** It enables fast retrieval of information based on its key.
- The great thing about hashing is, we can achieve all three operations (search, insert and delete) in O(1) time on average.
- Hashing is mainly used to implement a set of distinct items (only keys) and dictionaries (key value pairs).

Example


List = [11, 12, 13, 14, 15]
H (x) = [x % 10]
11%10
12%10
13%10
14%10
15%10
0
1
2
3
4
5
Hash Table
11
12
13
14
15
Introduction to Hashing

Component of Hashing



Key

- In the context of hashing, a "key" refers to the input data that's used by a hashing algorithm to produce a **hash value or hash code**.
- This hash value is then used to determine where to store or retrieve data within a hash table or other hashing structure.
- **Example:**
  - Imagine you want to store a list of names and their corresponding phone numbers. You can use the name as the key and the phone number as the value. A hash function would then map the name (key) to a specific index in a hash table where you can store the corresponding phone number.

Hash Function

- In Data Structures and Algorithms (DSA), a **hash function** is a mathematical function that converts a key (input value) into a hash value (output value).
- This hash value, often an index, is used to quickly store and retrieve data in a hash table, a specialized data structure.
- Example:
  - Imagine a simple hash table with 10 slots. A string "Bob" might have a hash function that calculates a hash value of 5. The "Bob" data would then be stored in slot 5 of the hash table.

Types of Hash Functions

- The primary types of **hash functions** are:
  - Division Method.
  - Mid Square Method.
  - Folding Method.
  - Multiplication Method.

1. **Division Method:**
   - The easiest and quickest way to create a hash value is through division. The k-value is divided by M in this hash function, and the result is used.Formula:
   - **h(K) = k mod M**
   - (where k = key value and M = the size of the hash table)

Types of Hash function

- **Mid Square Method**
  - The following steps are required to calculate this hash method:
  - k*k, or square the value of k
  - Using the middle r digits, calculate the hash value.
  - Formula:

    h(K) = h(k x k)

    (where k = key value)

- Example of Mid Square Method

```
k = 60Therefore,k = k x k
k = 60 x 60
k = 3600Thus,
h(60) = 60
```

Types of Hash Function

3. Folding Method

- The process involves two steps:
  - except for the last component, which may have fewer digits than the other parts, the key-value k should be divided into a predetermined number of pieces, such as k1, k2, k3,..., kn, each having the same amount of digits.
  - Add each element individually. The hash value is calculated without taking into account the final carry, if any.

- Formula:
  - **k = k1, k2, k3, k4, ..., kn**
  - **s = k1+ k2 + k3 + k4 +....+ kn**
  - **h(K)= s**
  - (Where, s = addition of the parts of key k)

```
k = 12345
k1 = 67; k2 = 89; k3 = 12Therefore,s = k1 + k2 + k3
s = 67 + 89 + 12
s = 168
```

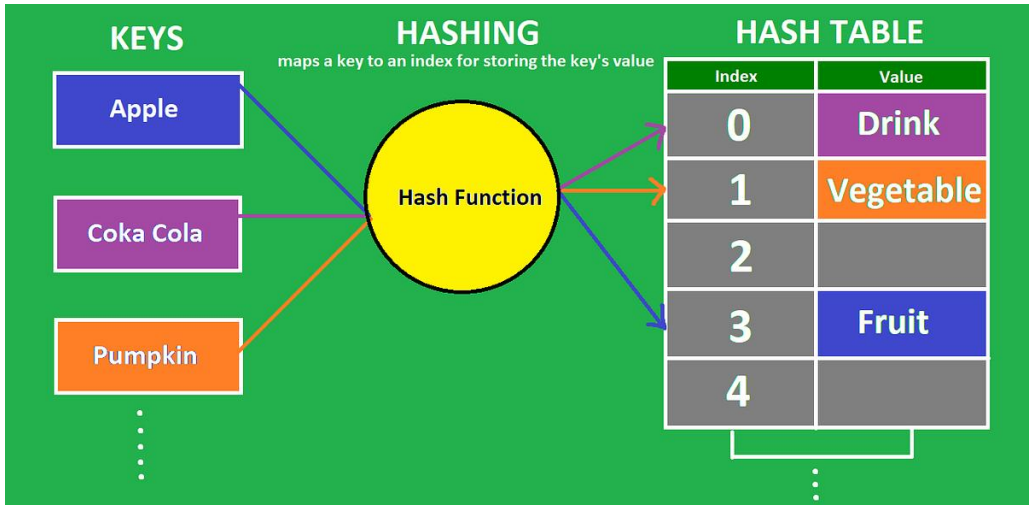
Types of Hash Function

4. Multiplication Method

- Determine a constant value. A, where (0, A, 1)
- Add A to the key value and multiply.
- Consider kA's fractional portion.
- Multiply the outcome of the preceding step by M, the hash table's size.
- Formula:
  - **h(K) = floor (M (kA mod 1))**
  - (Where, M = size of the hash table, k = key value and A = constant value)

Hash Table

- A **hash table** is a data structure that maps keys to values, using a hash function to determine the storage location of each key-value pair.
- This allows for very fast data access, with an average time complexity of O(1) for operations like insertion, deletion, and lookup.



What is a **Hash Collision**?

- **Collision** in a hash table is a term used to denote the phenomena when the hashing algorithm produces the same hash value for two or more keys using a hash function.
- However, it's crucial to note that collisions are not an issue; rather, they constitute a key component of hashing algorithms.
- Because various hashing methods used in data structures convert each input into a fixed-length code regardless of its length, collisions happen.
- The hashing algorithms will eventually yield repeating hashes since there are an infinite number of inputs and a finite number of outputs.

Collision Resolution Techniques in Data Structures

1. Open hashing/separate chaining/closed addressing
   - A typical collision handling technique called "separate chaining" links components with the same hash using linked lists. It is also known as closed addressing and employs **arrays** of **linked lists** to successfully prevent hash collisions.

- Advantages:
  - Implementation is simple and easy
  - We can add more keys to the table because the hash table has a lot of empty places.
  - Less sensitive than average to changing load factors
  - Typically utilized when there is uncertainty on the number and frequency of keys to be used in the hash table.

- Disadvantages:
  - Space is wasted
  - The length of the chain lengthens the search period.
  - Comparatively worse cache performance to closed hashing.

2. Closed hashing (Open addressing)

- Instead of using linked lists, open addressing stores each entry in the array itself. The hash value is not used to locate objects.
- To insert, it first verifies the array beginning from the hashed index and then searches for an empty slot using probing sequences.
- The probe sequence, with changing gaps between subsequent probes, is the process of progressing through entries.

Method for dealing with collisions in closed hashing

1) Linear Probing

- Linear probing includes inspecting the hash table sequentially from the very beginning. If the site requested is already occupied, a different one is searched. The distance between probes in linear probing is typically fixed (often set to a value of 1).

- Formula
  - **index = key % hashTableSize**

- Sequence

```
index = ( hash(n) % T)
(hash(n) + 1) % T
(hash(n) + 2) % T
(hash(n) + 3) % T … and so on.
```

Method for dealing with collisions in closed hashing

- **Quadratic Probing**
  - The distance between subsequent probes or entry slots is the only difference between linear and quadratic probing. You must begin traversing until you find an available hashed index slot for an entry record if the slot is already taken. By adding each succeeding value of any arbitrary polynomial in the original hashed index, the distance between slots is determined.

- Formula
  - **index = index % hashTableSize**

- Sequence

```
index = ( hash(n) % T)
(hash(n) + 1 x 1) % T
(hash(n) + 2 x 2) % T
(hash(n) + 3 x 3) % T … and so on
```

Method for dealing with collisions in closed hashing

3. Double-Hashing

- The time between probes is determined by yet another hash function. Double hashing is an optimized technique for decreasing clustering. The increments for the probing sequence are computed using an extra hash function.
- Formula
  - **(first hash(key) + i * secondHash(key)) % size of the table**
- Sequence

```
index = hash(x) % S
(hash(x) + 1*hash2(x)) % S
(hash(x) + 2*hash2(x)) % S
(hash(x) + 3*hash2(x)) % S … and so on
```

Importance of Hashing

- Easy retrieval of required information from large data sets in an efficient manner.
- The hash code produced by the hash function serves as the unique identifier in the data set thus maintaining data integrity.
- The data is stored in a structured manner as there is an index for every record in the hash table. This ensures efficient storage and retrieval.

Limitations of Hashing

- Many a time there leads to a situation of collision where two or more inputs have the same hash value.
- The performance of the hashing algorithm depends upon the quality of the hash function. Sometimes, a not well-thought-of hash function may lead to collisions thus reducing the efficiency of the algorithm.

Efficiency Comparison of Searching Algorithm

Time Complexity

Searching Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Interpolation Search	O(1)	O(log log n)	O(n)
Exponential Search	O(1)	O(log n)	O(log n)
Jump Search	O(1)	O(√n)	O(√n)
Fibonacci Search	O(1)	O(log n)	O(log n)
Ternary Search	O(1)	O(log n)	O(log n

Space Complexity

Searching Algorithm	Space Complexity
Linear Search	O(1)
Binary Search	O(1) (iterative) / O(log n) (recursive)
Interpolation Search	O(1)
Exponential Search	O(1)
Jump Search	O(1)
Fibonacci Search	O(1)
Ternary Search	O(1) (iterative) / O(log n) (recursive)

Presented By: Saroj Bhandari

Thank you !!!